SOFTWARE REQUIREMENTS SPECIFICATION

Employee Group Insurance (EGI)

- **V. Hemanth Kumar**

Table of Contents

1. Introduction 3
   1.1 Purpose 3
   1.2 Scope 3
   1.3 Definitions and Acronyms 3
   1.4 References 4
2. Overall Description 5
   2.1 Product Perspective 5
   2.2 Product Functions 5
   2.3 User Classes and Characteristics 6
   2.4 Operating Environment 7
   2.5 Design and Implementation Constraints 7
3. System Architecture 8
   3.1 Architectural Pattern 8
   3.2 Technology Stack 9
   3.3 Database Design 9
4. Core Business Workflows & Rules 11
   4.1 Corporate Onboarding & Verification 11
   4.2 Authentication, Security & Passwords 11
   4.3 Invoicing & Automated Penalty System 12
   4.4 Agent Commissions 12
   4.5 Endorsements & Prorated Premiums 13
   4.6 Claim Validation & Settlement 13
   4.7 Dependent Management 14
   4.8 Notification & Polling System 14
   4.9 Background Automation Lifecycle 14
   4.10 Audit Logging & System Analytics 14
5. Specific Requirements 15
   5.1 Functional Requirements 15
   5.2 Non-Functional Requirements 17
6. System Features 19
   6.1 User Authentication and Authorization 19
   6.2 Insurance Plan Management 19
   6.3 Policy Management 19
   6.4 Claims Management 19
   6.5 Policy Endorsements 20
   6.6 Dashboard and Reporting 20
7. External Interface Requirements 21
8. Security Requirements 22
9. Quality Attributes 23
10. Appendices 24

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document provides a comprehensive description of the Employee Group Insurance (EGI) Portal system. It details the functional and non-functional requirements, system architecture, core business rules, and design constraints. This document is intended for developers, project managers, testers, and stakeholders involved in the development and deployment of the EGI Portal.

1.2 Scope

The EGI Portal is a web application designed to digitize and streamline group health and employee insurance operations. It manages the complete lifecycle of corporate insurance policies — from initial client onboarding and policy creation to premium invoicing, policy endorsements (revisions), and client claims settlement.

Primary Objectives:

- Eliminate manual paperwork in corporate insurance management.
- Provide self-service dashboards for employers to manage their employee rosters.
- Automate premium calculations, penalty applications, and agent commissions.
- Provide a robust workflow for claims adjudication.
- Multi-role user management (Admin, Agent, Customer, Claims Officer).
- Dynamic insurance plan creation and management.
- Member and dependent enrollment with bulk upload support.
- Automated invoice generation, payment tracking, and policy suspension.
- Comprehensive audit logging for compliance and non-repudiation.

1.3 Definitions and Acronyms

Term	Definition
EGI	Employee Group Insurance
SRS	Software Requirements Specification
API	Application Programming Interface
JWT	JSON Web Token
RBAC	Role-Based Access Control
EF Core	Entity Framework Core
DTO	Data Transfer Object
SOLID	Single Responsibility, Open-Closed, Liskov Substitution, Interface Segregation, Dependency Inversion
KYB / KYC	Know Your Business / Know Your Customer — identity verification for corporate clients

Member	Primary employee enrolled in an insurance policy
Dependent	Family member covered under a member's policy
Endorsement	Request to modify an existing active policy (e.g., adding/removing members)
Policy Assignment	Association of an insurance plan with a corporate client
Claim	Request for reimbursement for covered medical expenses
Premium	Amount paid periodically for insurance coverage
Sum Insured	Maximum claimable amount per member/dependent in a policy year
Prorated Premium	Partial premium calculated proportionally to remaining coverage days
Grace Period	7-day window after invoice due date before penalty is applied
UnitOfWork	Design pattern ensuring transactional consistency across database operations
QuestPDF	PDF generation library used for settlement and invoice documents

1.4 References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications
- Clean Architecture — Robert C. Martin
- ASP.NET Core Documentation: https://docs.microsoft.com/aspnet/core
- Entity Framework Core Documentation: https://docs.microsoft.com/ef/core
- Angular Documentation: https://angular.io/docs
- JWT Authentication Best Practices: https://tools.ietf.org/html/rfc7519
- QuestPDF Documentation: https://www.questpdf.com

2. Overall Description

2.1 Product Perspective

The EGI Portal is a standalone web-based system serving as a centralized platform for managing group insurance operations. The system consists of:

- Frontend Application: Angular v17+ (Standalone Components, Signals) — responsive web interface.
- Backend API: ASP.NET Core Web API (.NET 8) providing RESTful services following Clean Architecture.
- Database: Microsoft SQL Server with Entity Framework Core (Code-First).
- Document Storage: File-system-based management for KYC documents and claim attachments.
- Email Service: Automated SMTP-based notifications for system events.
- Background Services: .NET Hosted Services (IHostedService) for automated daily tasks — invoice generation, overdue marking, policy suspension, and expiry.
- PDF Reports: QuestPDF for dynamically generating Invoice PDFs and Claim Settlement Documents.

2.2 Product Functions

The EGI Portal provides the following major function groups:

Function Group	Description
User Management	Registration, JWT authentication, RBAC, password management, staff provisioning, KYB guard.
Insurance Plan Management	Admin creates master plans (Silver, Gold, Platinum) with coverage types and premium rules.
Corporate Client Management	KYB document upload, admin approval, least-loaded agent auto-assignment.
Policy Management	Plan assignment to clients, Excel bulk member enrollment, dependent management.
Claims Processing	Claim submission, concurrency-locked adjudication, settlement PDF generation, limit deduction.
Policy Endorsements	Mid-term roster changes with prorated premium adjustments and agent approval.
Invoice & Payment	Automated generation, partial payments (min 50%), overdue penalties, policy suspension.
Agent Commissions	Dynamic rate commissions (2%–8%) calculated strictly on payment receipt.
Dashboard & Analytics	Role-specific Chart.js dashboards with real-time telemetry and audit analytics.
Audit Logging	Immutable ledger of all write mutations — UserId, Timestamp, Entity, Action.
Notifications	Frontend polling-based notification system with isRead tracking.
Background Automation	Daily scheduled workers for invoice generation, penalty, suspension, and expiry.

2.3 User Classes and Characteristics

Role 1: Administrator (Admin)

The overarching system manager with full visibility and configuration rights.

- Technical Expertise: High
- Primary Responsibilities: System configuration, staff management, insurance plan creation, corporate client KYB approval.
- Key Capabilities: Creates master Insurance Plans; provisions Claims Officer accounts; approves/rejects corporate clients (up to 3 rejections before permanent block); has read-only global ledger access to all policies, claims, invoices, and audit logs; views system-wide Chart.js analytics.

Role 2: Agent (Sales Intermediary)

The sales representative mapping corporate clients to insurance products.

- Technical Expertise: Medium
- Primary Responsibilities: Corporate client onboarding, policy assignment, endorsement verification, commission tracking.
- Key Capabilities: Creates customer accounts; assigns Insurance Plans to approved clients; reviews and approves/rejects endorsement requests; earns commissions dynamically calculated on payment receipt (2%–8% based on company size).

Role 3: Customer (Corporate Employer / HR Manager)

The policyholder managing insurance for their organisation.

- Technical Expertise: Low to Medium
- Primary Responsibilities: KYB completion, member/dependent roster management, invoice payment, claim submission.
- Key Capabilities: Uploads KYC documents for Admin verification; performs Excel bulk uploads for employee enrollment; views and pays auto-generated invoices (partial payments with 50% floor); submits medical claims on behalf of employees; requests policy endorsements as workforce changes.

Role 4: Claims Officer (Adjudicator)

The back-office specialist reviewing and adjudicating medical claims.

- Technical Expertise: Medium
- Primary Responsibilities: Claim queue management, concurrency-locked adjudication.
- Key Capabilities: Views a live queue of all submitted claims; acquires a concurrency lock ('Take') before adjudicating to prevent double-processing; can release a claim back to the pool; approves (with sanctioned amount) or rejects (with mandatory reason) claims; deduction of approved amount from member's Sum Insured.

2.4 Operating Environment

- Client-Side: Modern web browsers (Chrome, Firefox, Edge, Safari) with JavaScript enabled — supports Angular v17+.
- Server-Side: Windows Server or Linux with .NET 8 runtime.

- Database: Microsoft SQL Server 2019 or later.
- Web Server: IIS or Kestrel.
- Network: Internet connectivity required for all users.
- Mobile Support: Responsive design (Tailwind CSS) compatible with tablets and smartphones.

2.5 Design and Implementation Constraints

- Architecture: Must follow Clean Architecture (Domain → Application → Infrastructure → WebAPI) with strict layer isolation.
- Security: JWT-based authentication bounded tightly to role claims; all endpoints protected.
- State Management: Angular Signals for frontend reactive state; no shared mutable globals.
- Frontend Architecture: Lazy-loaded routes; strictly decoupled API services per role (admin.service.ts, customer.service.ts, etc.).
- Database: Entity Framework Core Code-First only; direct SQL queries prohibited.
- Background Jobs: .NET IHostedService for all scheduled automations.
- PDF Generation: QuestPDF only for Invoice and Settlement PDFs.
- Testing: Minimum 80% code coverage for critical business logic; xUnit + Moq.
- Performance: API response time under 2 seconds for 95% of requests; bulk uploads up to 1,000 records.
- Compliance: Immutable audit logs retained for minimum 7 years.

3. System Architecture

3.1 Architectural Pattern — Clean Architecture

The EGI Portal implements Clean Architecture, emphasizing separation of concerns and independence from frameworks, UI, and databases. Dependencies flow strictly inward.

Domain Layer (Core)

- Contains business entities and domain logic. No dependencies on other layers.
- Entities: User, CorporateClient, InsurancePlan, PlanCoverage, PolicyAssignment, Member, Dependent, Claim, ClaimDocument, PolicyEndorsement, Invoice, Payment, AuditLog, Notification, AgentCustomer, CorporateClientDocument (16 entities).
- Enums: UserRole, UserStatus, ClaimStatus, EndorsementStatus, EndorsementType, PolicyStatus, InvoiceStatus, PaymentStatus, PaymentMethod, CoverageType, CoveredGroup, ClaimDocumentType, DocumentType, Gender, RelationshipType, BillingFrequency, BusinessCategory, VerificationStatus (18 enums).

Application Layer

- Contains business rules, use cases, service interfaces, DTOs, and business logic implementations.
- Services: AuthService, ClaimService, PolicyEndorsementService, InsurancePlanService, InvoiceService, MemberService, NotificationService, AuditLogService, CommissionService, and more (12+ implementations, 20+ interfaces, 30+ DTOs).
- Patterns: Repository Pattern, Unit of Work Pattern.

Infrastructure Layer

- Implements interfaces defined in the Application Layer. Handles external concerns.
- Components: EF Core repositories, JWT token generator, BCrypt password hasher, SMTP email service, document file storage.
- Database Context: EGIDbContext with full entity configurations.
- Background Service: InsuranceAutomationWorker (IHostedService) running daily scheduled sweeps.

WebAPI Layer (Presentation)

- Exposes RESTful API endpoints, handles HTTP requests/responses.
- Controllers: AuthController, AdminDashboardController, AgentDashboardController, CustomerDashboardController, ClaimsOfficerDashboardController, and feature controllers (6+ controllers).
- Middleware: GlobalExceptionHandler, RequestLoggingMiddleware.
- Filters: ApiResponseFilter for consistent JSON response envelope formatting.

3.2 Technology Stack

Component	Technology / Detail
Frontend Framework	Angular v17+ — Standalone Components, Signals for state management, Lazy-loaded routes, TypeScript
Frontend Styling	Tailwind CSS — Utility-first, responsive design
Frontend Data Viz	Chart.js (via ng2-charts) — Dashboard analytics and telemetry charts
Backend Framework	ASP.NET Core Web API (.NET 8) — RESTful services
Backend Language	C# — Strongly typed, async/await throughout
ORM	Entity Framework Core 8.x — Code-First migrations, Repository pattern
Database	Microsoft SQL Server — Relational, ACID-compliant
Authentication	JWT (JSON Web Tokens) with BCrypt password hashing
PDF Generation	QuestPDF — Invoice PDFs and Claim Settlement Documents
Background Jobs	.NET IHostedService — Daily automated worker for invoicing, penalties, suspensions
Testing Framework	xUnit with Moq and EF Core In-Memory provider
API Documentation	Swagger / OpenAPI 3.0
Dependency Injection	Built-in ASP.NET Core DI Container
Object Mapping	AutoMapper — DTO ↔ Domain entity mapping
Logging	ASP.NET Core Logging with custom request logging middleware

3.3 Database Design — Key Entities & Relationships

Entity	Description	Key Relationships
User	System users across all roles	One-to-Many with Claims, PolicyEndorsements, AuditLogs, Notifications
InsurancePlan	Master insurance product definitions	One-to-Many with PlanCoverage, PolicyAssignment
PlanCoverage	Coverage details per plan (type, limit, group)	Many-to-One with InsurancePlan
CorporateClient	Corporate customer profiles and KYB data	One-to-Many with PolicyAssignment, CorporateClientDocument
AgentCustomer	Agent ↔ Customer mapping table	Many-to-One with User (Agent), CorporateClient
PolicyAssignment	Policy issued to a corporate client	Many-to-One with InsurancePlan, CorporateClient; One-to-Many with Member, Invoice, Endorsement

Member	Primary enrolled employees	Many-to-One with PolicyAssignment; One-to-Many with Dependent, Claim
Dependent	Family members of employees	Many-to-One with Member; One-to-Many with Claim (draws from Member's Sum Insured)
Claim	Insurance claim adjudication packets	Many-to-One with PolicyAssignment, Member, Dependent, User (Reviewer)
ClaimDocument	Supporting documents per claim	Many-to-One with Claim
PolicyEndorsement	Mid-term policy modification requests	Many-to-One with PolicyAssignment, User (Requester, Reviewer)
Invoice	Billing invoices per policy cycle	Many-to-One with PolicyAssignment; One-to-Many with Payment
Payment	Payment transactions against invoices	Many-to-One with Invoice
CorporateClientDocument	KYB/KYC documents for verification	Many-to-One with CorporateClient
Notification	System event notifications per user	Many-to-One with User (tracks isRead)
AuditLog	Global immutable ledger of all mutations	Many-to-One with User (UserId, Timestamp, Entity, Action)

4. Core Business Workflows & Rules

This section documents all critical business logic rules that drive the EGI Portal, directly derived from the system context document. These rules must be implemented exactly as specified.

4.1 Corporate Onboarding & Verification

1. A corporate user registers via the self-registration endpoint.
2. They complete their company profile and upload mandatory KYB documents (Company Registration, Tax IDs, etc.).
3. An Admin manually reviews the uploaded documents. The Admin can approve or reject the submission.
4. Rejection Limit: Corporate clients may be rejected up to 3 times. On the 3rd rejection, the account is permanently blocked with no further appeal path.
5. Auto-Assignment on Approval: Upon Admin approval, the system algorithmically assigns the Customer to the Agent with the lowest current client load ('least loaded' agent selection).

Business Rule	Accounts that have not completed KYB submission are blocked by an ‘Account Incomplete’ dashboard guard — they cannot access any dashboards until their KYB submission is explicitly Approved by an Admin.

4.2 Authentication, Security & Passwords

- All API interactions are strictly protected by JWT Authentication bounded to the user's explicit Role claim.
- First Login Forfeiture: New users provisioned by an Admin (Claims Officer) or an Agent (Customer) are forced to change their default password on first login via a requirePasswordChange flag before accessing any dashboard.
- Forgot Password: Users can reset passwords via an emailed security token reset process.
- KYB Guard: B2B Corporate Clients are blocked by an ‘Account Incomplete’ dashboard guard until their KYB submission is explicitly Approved by an Admin.
- Role Isolation: JWT role claims strictly control data and UI access — each role sees only its own data perimeter.

4.3 Invoicing & Automated Penalty System

Invoices are generated dynamically based on the policy's BillingFrequency (Monthly or Annually).

Timeline	System Behaviour
Day 0	Invoice is generated automatically by the background worker. Status: Pending.
Day 1–7 (Grace Period)	Customer can pay without any penalty. Invoice remains Pending.
Day 8	Background worker detects overdue status. Applies a 10% financial penalty to the invoice amount.
Day 15	If still unpaid, the background worker automatically flags the Policy as Suspended — instantly blocking all new claim submissions for that company.

Partial Payments Constraint:

- Customers may pay off an invoice in multiple chunks (partial payments are natively supported).
- Hard Rule: Any single partial payment must clear at least 50% of the total invoice amount. Payments below this threshold are rejected.

Annual Billing Discount:

- Policies opted for **Annual Billing** receive an automatic **5% discount** on the total calculated premium. This discount is applied by the system automatically at the point of invoice generation when the policy's BillingFrequency is set to **Annually**. Monthly policies do not receive this discount.

4.4 Agent Commissions

- Trigger: Commissions are calculated strictly upon receipt of payment — NOT upon policy creation or assignment.
- Dynamic Rates: The commission rate scales based on the number of insured lives in the corporate client:

Company Size (Insured Lives)	Commission Rate
Small Business (≤ 50 lives)	8%
Medium Business (51–250 lives)	6%
Large Business (251–1000 lives)	4%
Enterprise (> 1000 lives)	2%

4.5 Endorsements (Policy Revisions) & Prorated Premiums

When a company hires or terminates an employee mid-policy year, they submit an Endorsement Request.

- Endorsement Types: AddMember, RemoveMember, AddDependent, RemoveDependent.
- The system calculates a Prorated Premium Adjustment down to the exact day of the change.
- Example: If a new employee is added 6 months (183 days) into a 365-day annual policy, the system generates an adjustment invoice charging 50% (183/365) of the annual premium for that specific member.
- All endorsements are reviewed and approved/rejected by the assigned Agent (or Admin).
- Upon approval, the system applies the roster change to the policy and triggers any premium adjustment invoice.
- Credit on Removal: For **RemoveMember** and **RemoveDependent** endorsements, the system computes the unused coverage days remaining in the policy period and issues a prorated Financial Credit. This credit is automatically applied against the customer's next generated invoice, reducing the amount owed.

Formula	Prorated Premium = (Annual Premium per Member) × (Remaining Days in Policy Period / Total Days in Policy Period)

- Dependent Premium Multipliers: Each insured person type carries a specific multiplier applied against the policy's Base Premium when calculating endorsement adjustment invoices:

Insured Person Type	Premium Multiplier (× Base Premium)
Primary Member (Employee)	1.0× (Base Premium)
Spouse	0.8× (Base Premium)
Child	0.4× (Base Premium)
Parent	1.2× (Base Premium)

4.6 Claim Validation & Settlement

- Hard Validation: A claim CANNOT be submitted if the policy's invoice is overdue past the 15-day suspension threshold.
- Adjudication Lock: A Claims Officer MUST explicitly 'Lock/Take' a claim before adjudicating. This prevents concurrent double-processing by multiple officers.
- Release: If an officer cannot complete adjudication, they can 'Release' the claim back to the open queue.
- Approval: When a Claim is approved, the sanctioned amount is permanently deducted from the Member's (or Dependent's primary Member's) remaining Sum Insured for that policy year.
- Rejection Rule: Any transition to 'Rejected' status requires a mandatory Reason string — the system enforces this at the API level.
- Settlement PDF: Upon approval, the system dynamically generates a PDF Settlement Document using QuestPDF and stores it for the customer to download.
- Dependent Claims: Claims submitted for a Dependent draw down from the primary Member's unified family Sum Insured limit — ensuring the total family liability never exceeds the policy's cap.

4.7 Dependent Management

- Primary Members (employees) can have Dependents enrolled: Spouse, Children, Parents.
- Coverage Groups: The Insurance Plan defines which group types are covered — 'Employee Only', 'Employee & Family', or 'Employee, Family & Parents'.
- Dependent claims draw from the primary Member's unified Sum Insured pool.
- Dependents are enrolled and modified via Endorsements (mid-policy) or initial bulk upload.

4.8 Notification & Polling System

- The Angular frontend features an active polling mechanism that periodically hits backend notification endpoints.
- Key Notification Triggers:
  - Customer: Invoice overdue, Endorsement approved/rejected, Claim settled.
  - Agent: New customer auto-assigned to them.
  - Claims Officer: New claim submitted in queue.
- Notifications track isRead status and surface in a header dropdown UI.

4.9 Background Automation Lifecycle (Daily Worker)

The InsuranceAutomationWorker (IHostedService) runs daily sweeps executing three jobs in sequence:

Job Name	Description	Effect
ExpireOldPoliciesAsync	Scans all Active policies. Flags any that have crossed their EndDate as Expired.	PolicyStatus → Expired
GenerateDueInvoicesAsync	Checks BillingFrequency (Monthly/Annual) for all active policies. Generates the next invoice exactly when due.	New Invoice record created → Pending
MarkOverdueInvoicesAsync	Scans unpaid invoices. Fires 10% penalty at Day 8. Fires Policy Suspension at Day 15.	InvoiceStatus → Overdue; PolicyStatus → Suspended

4.10 Audit Logging & System Analytics

- Audit Ledger: Every write/update mutation inside the EGI Platform creates an immutable AuditLog entry.
- Fields captured: UserId, Timestamp, EntityName, EntityId, Action performed.
- Scope: Covers all critical mutations — policy changes, claim approvals, endorsement reviews, user provisioning.
- Dashboard Telemetry: The Admin dashboard aggregates audit data and renders Chart.js visualizations including 'Claim Transmission Trends', active liabilities, and pending workflow counts.
- Retention: Audit logs must be retained for a minimum of 7 years for regulatory compliance.

5. Specific Requirements

5.1 Functional Requirements

FR-1: User Authentication

- **FR-1.1:** System shall allow users to register with email and password.
- **FR-1.2:** System shall authenticate users using JWT tokens with role claims.
- **FR-1.3:** System shall support password change functionality for all authenticated users.
- **FR-1.4:** System shall provide 'Forgot Password' via emailed security token reset process.
- **FR-1.5:** System shall enforce a requirePasswordChange flag for Admin/Agent-provisioned users on first login.
- **FR-1.6:** System shall track last login timestamp per user.

FR-2: User Authorization & Access Control

- **FR-2.1:** System shall implement strict RBAC (Admin, Agent, Customer, ClaimsOfficer) via JWT role claims.
- **FR-2.2:** System shall restrict all API endpoints based on user roles using [Authorize] attributes.
- **FR-2.3:** Admin shall be able to create staff accounts (Agent, ClaimsOfficer).
- **FR-2.4:** Corporate clients shall be blocked by an ‘Account Incomplete’ dashboard guard until their KYB submission is explicitly Approved by an Admin.
- **FR-2.5:** System shall enforce data perimeter isolation — each role sees only its own tenant data.

FR-3: Insurance Plan Management (Admin)

- **FR-3.1:** Admin shall create insurance plans with PlanCode, PlanName, BasePremium, and description.
- **FR-3.2:** Admin shall define multiple coverage types per plan (Health, Life, Accident, Critical Illness).
- **FR-3.3:** Admin shall set coverage limits and covered groups for each coverage type.
- **FR-3.4:** Admin shall activate or deactivate insurance plans.
- **FR-3.5:** System shall maintain plan creation and modification history via AuditLog.

FR-4: Corporate Client Management & KYB

- **FR-4.1:** Agents shall be able to register corporate clients with company details (Name, RegistrationNumber, BusinessCategory).
- **FR-4.2:** Corporate clients shall upload mandatory KYB documents (Company Registration, Tax IDs).
- **FR-4.3:** Admin shall review and approve or reject corporate client KYB submissions.
- **FR-4.4:** System shall permanently block a corporate account after 3 KYB rejections.
- **FR-4.5:** System shall auto-assign approved clients to the least-loaded Agent algorithmically.

FR-5: Policy Assignment

- **FR-5.1:** Agents shall assign approved Insurance Plans to approved corporate clients.
- **FR-5.2:** System shall set policy start and end dates and BillingFrequency (Monthly/Annual).
- **FR-5.3:** System shall track policy status (Active, Expired, Suspended).
- **FR-5.4:** System shall support multiple policies per corporate client.

FR-6: Member & Dependent Enrollment

- **FR-6.1:** Corporate clients shall enroll members individually or via Excel bulk upload.
- **FR-6.2:** Bulk upload shall support up to 1,000 member records per file.
- **FR-6.3:** System shall validate member data (name, email, date of birth, gender).
- **FR-6.4:** Members shall enroll dependents with relationship type (Spouse, Child, Parent, Sibling).
- **FR-6.5:** Enrolled dependents shall draw from the primary Member's unified Sum Insured pool.

FR-7: Claims Management

- **FR-7.1:** Corporate clients shall submit claims for members or dependents with amount, type, and reason.
- **FR-7.2:** System shall reject claim submission if the policy is Suspended (overdue past Day 15).
- **FR-7.3:** System shall generate a unique ClaimNumber for every submitted claim.
- **FR-7.4:** System shall allow upload of supporting documents (bills, prescriptions, lab reports).
- **FR-7.5:** Claims Officers shall 'Take' (lock) a claim before adjudication to prevent concurrent processing.
- **FR-7.6:** Claims Officers shall be able to 'Release' a locked claim back to the open queue.
- **FR-7.7:** Claims Officers shall approve claims with a sanctioned amount, or reject with a mandatory reason string.
- **FR-7.8:** System shall permanently deduct approved claim amount from the Member's remaining Sum Insured.
- **FR-7.9:** System shall generate a QuestPDF Settlement Document upon approval.
- **FR-7.10:** System shall send notifications on claim status transitions.

FR-8: Policy Endorsements

- **FR-8.1:** Corporate clients shall submit endorsement requests (AddMember, RemoveMember, AddDependent, RemoveDependent).
- **FR-8.2:** System shall calculate a prorated premium adjustment to the exact day for each endorsement. For AddMember/AddDependent endorsements, an adjustment invoice is generated for the remaining coverage days. For RemoveMember/RemoveDependent endorsements, the system shall calculate the unused coverage days and issue a prorated Financial Credit, automatically applied against the customer’s next generated invoice.
- **FR-8.3:** Endorsement data shall be stored as JSON for agent review.
- **FR-8.4:** Agents (or Admin) shall approve or reject endorsements.
- **FR-8.5:** Approved endorsements shall trigger policy roster update and adjustment invoice generation.
- **FR-8.6:** System shall notify customers on endorsement status changes.

FR-9: Invoice & Payment Management

- **FR-9.1:** Background worker shall automatically generate invoices per BillingFrequency cycle.
- **FR-9.2:** System shall apply a 10% penalty to invoices unpaid after the 7-day grace period (Day 8).
- **FR-9.3:** System shall automatically suspend the policy if invoice remains unpaid at Day 15.
- **FR-9.4:** System shall accept partial payments but enforce a minimum of 50% of total invoice amount per payment.
- **FR-9.5:** System shall track invoice status: Pending, Paid, Overdue, Cancelled.
- **FR-9.6:** Agent commissions shall be calculated strictly upon payment receipt at dynamic rates (2%–8%) based on insured lives.

FR-10: Dashboard & Reporting

- **FR-10.1:** Admin Dashboard: System-wide metrics, claim trends (Chart.js), pending approvals, audit log access.
- **FR-10.2:** Agent Dashboard: Customer count, policies sold, commissions earned, pending premiums.
- **FR-10.3:** Customer Dashboard: Policy count, member count, pending claims, unpaid invoices.
- **FR-10.4:** Claims Officer Dashboard: Live claim queue, pending/approved/rejected counts, processing stats.
- **FR-10.5:** All dashboards shall reflect real-time data via API polling.

FR-11: Notification System

- **FR-11.1:** Frontend shall poll backend notification endpoints at regular intervals.
- **FR-11.2:** System shall create notifications for: invoice overdue, endorsement approved/rejected, claim settled, customer assigned to agent.
- **FR-11.3:** Notifications shall track isRead status and render in a header dropdown UI component.

FR-12: Audit Logging

- **FR-12.1:** System shall create an immutable AuditLog entry for every write/update mutation.
- **FR-12.2:** Each AuditLog entry shall capture: UserId, Timestamp, EntityName, EntityId, Action.
- **FR-12.3:** Audit logs shall be retained for a minimum of 7 years.
- **FR-12.4:** Admin dashboard shall aggregate and visualise audit data via Chart.js.

5.2 Non-Functional Requirements

NFR-1: Performance

- **NFR-1.1:** System shall respond to 95% of API requests within 2 seconds under normal load.
- **NFR-1.2:** System shall support at least 1,000 concurrent users.
- **NFR-1.3:** Database queries shall be optimized with proper indexing on foreign keys and frequently filtered columns.
- **NFR-1.4:** System shall handle bulk member uploads of up to 1,000 records within acceptable time.
- **NFR-1.5:** Dashboard data shall load within 3 seconds.

NFR-2: Security

- **NFR-2.1:** System shall use JWT tokens with expiration for all API authentication.
- **NFR-2.2:** Passwords shall be hashed using BCrypt with salt.
- **NFR-2.3:** System shall enforce HTTPS for all client-server communications.
- **NFR-2.4:** Role-based authorization shall be enforced at the API endpoint level using JWT role claims.
- **NFR-2.5:** System shall validate and sanitize all user inputs to prevent XSS and SQL injection.
- **NFR-2.6:** EF Core parameterized queries shall prevent SQL injection attacks.
- **NFR-2.7:** System shall log all security-relevant events (logins, failed attempts, privilege escalation attempts).

NFR-3: Reliability

- **NFR-3.1:** System shall maintain 99.5% uptime.

- **NFR-3.2:** Unit of Work pattern shall ensure transactional data consistency across all mutations.
- **NFR-3.3:** Global exception handler middleware shall manage all unhandled errors gracefully.
- **NFR-3.4:** Database integrity shall be enforced via EF Core foreign key constraints.

NFR-4: Usability

- **NFR-4.1:** User interface shall be intuitive with role-specific navigation.
- **NFR-4.2:** System shall provide clear inline validation feedback on all forms.
- **NFR-4.3:** System shall be fully responsive across desktop, tablet, and mobile (Tailwind CSS).
- **NFR-4.4:** API documentation shall be available at runtime via Swagger UI.

NFR-5: Maintainability

- **NFR-5.1:** Codebase shall follow Clean Architecture and SOLID principles.
- **NFR-5.2:** Minimum 80% test coverage for critical business logic.
- **NFR-5.3:** All service interfaces and methods shall have XML documentation comments.
- **NFR-5.4:** Dependency injection shall be used throughout for loose coupling and testability.

NFR-6: Scalability

- **NFR-6.1:** Stateless API design shall support horizontal scaling behind a load balancer.
- **NFR-6.2:** Background worker design shall support distributed execution.
- **NFR-6.3:** Database schema shall support partitioning strategies for large datasets.

NFR-7: Compliance

- **NFR-7.1:** Immutable AuditLog shall cover all critical system mutations for non-repudiation.
- **NFR-7.2:** System shall comply with applicable data protection regulations (GDPR/PDPA/local equivalents).
- **NFR-7.3:** Audit logs shall be retained for a minimum of 7 years.

6. System Features

6.1 User Authentication and Authorization

Description: Secure user authentication and strict role-based access control via JWT role claims.

Priority: Critical

Key Capabilities:

- User registration with email validation and password hashing (BCrypt).
- JWT token generation on login with embedded role claims.
- requirePasswordChange enforcement on first login for provisioned users.
- Forgot Password via emailed security token.
- Account Incomplete guard for KYB-incomplete corporate clients.
- Role-specific navigation — each role sees only its allowed UI and data.

6.2 Insurance Plan Management

Description: Admin-controlled configuration of master insurance product offerings.

Priority: Critical

Key Capabilities:

- Create plans with unique PlanCode, PlanName, BasePremium (e.g., Silver, Gold, Platinum).
- Define multiple coverage types per plan: Health, Life, Accident, Critical Illness.
- Set CoverageLimit and CoveredGroup per coverage type.
- Activate or deactivate plans (inactive plans cannot be newly assigned).
- All plan modifications create AuditLog entries.

6.3 Policy Management

Description: Policy assignment, member/dependent enrollment, and lifecycle management.

Priority: Critical

Key Capabilities:

- Agents assign Insurance Plans to approved corporate clients.
- Corporate clients enroll members individually or via Excel bulk upload (up to 1,000 records).
- Dependents are enrolled under primary members with relationship type.
- Policy lifecycle tracked: Active → Suspended (if overdue) → Expired.
- Background worker auto-expires policies past their EndDate daily.

6.4 Claims Management

Description: End-to-end claim submission, concurrency-locked adjudication, and settlement.

Priority: Critical

Key Capabilities:

- Corporate clients submit claims for members or dependents with documents.
- Hard block on claim submission if policy is Suspended (Day 15+ overdue).
- Claims Officers acquire an exclusive lock ('Take') before adjudicating.

- Officers may release a lock ('Release') without adjudicating.
- Approval deducts exact amount from Member's (or Dependent's primary Member's) Sum Insured.
- Rejection requires a mandatory Reason string.
- QuestPDF Settlement Document generated and stored on approval.
- Claim status flow: Pending → InReview → Approved / Rejected.

6.5 Policy Endorsements

Description: Mid-term policy modifications with prorated premium adjustments.

Priority: High

Key Capabilities:

- Customers submit endorsement requests: AddMember, RemoveMember, AddDependent, RemoveDependent.
- System computes prorated premium adjustment to the exact day.
- Agents review endorsement data (stored as JSON) and approve or reject.
- Approved endorsements trigger roster update and adjustment invoice.
- Full audit trail for all endorsement activities.

6.6 Dashboard and Reporting

Description: Role-specific real-time dashboards with Chart.js analytics.

Priority: High

Role-specific dashboard contents:

- Admin: System-wide metrics, claim transmission trends (Chart.js), active liabilities, pending KYB approvals, full audit ledger access.
- Agent: Customer count, policies sold, total commissions earned, pending premium payments.
- Customer: Policy count, member/dependent count, pending claims, outstanding invoices.
- Claims Officer: Live claim queue size, claims approved vs. rejected stats, average processing time.

7. External Interface Requirements

7.1 User Interfaces

- Responsive Angular v17+ SPA with tab-based modular dashboard design.
- Role-specific navigation menus and guarded routes (Angular Route Guards).
- Form validation with real-time inline feedback (Angular Reactive Forms).
- Data tables with sorting, filtering, and pagination.
- File upload interface with type and size validation.
- Header-mounted notification dropdown with isRead tracking.
- Chart.js dashboard widgets for real-time analytics.
- Tailwind CSS utility classes for consistent responsive styling.

7.2 Hardware Interfaces

The system has no direct hardware interfaces. It operates through standard web browsers and network infrastructure accessible on any device with a modern browser.

7.3 Software Interfaces

Interface	Technology	Purpose
Database Management System	Microsoft SQL Server 2019+	Data persistence via Entity Framework Core
Email Service	SMTP Server	Password reset tokens and system notifications
PDF Engine	QuestPDF (.NET library)	Invoice and Claim Settlement PDF generation
Background Job Runtime	.NET IHostedService	Daily automated worker scheduling
File System	Local or Network Storage	KYC/KYB documents and claim attachments
Web Browser	Chrome, Firefox, Edge, Safari	Client-side Angular SPA rendering
Operating System	Windows Server or Linux	Server hosting and .NET 8 runtime

7.4 Communication Interfaces

- HTTPS: All client-server communication is encrypted via TLS/HTTPS.
- REST API: RESTful architecture for all API endpoints with JSON request/response bodies.
- JSON: Data exchange format for all API interactions.
- SMTP: Outbound email for password reset tokens and notifications.
- HTTP Polling: Angular frontend polls notification endpoints at intervals for real-time alerts.
- WebSocket / SignalR (Future): Planned for true real-time push notifications.

8. Security Requirements

8.1 Authentication

- JWT-based token authentication on all protected endpoints.
- Token expiration after configurable period (default: 1 hour).
- requirePasswordChange flag enforced for provisioned users on first login.
- BCrypt hashing with salt for all stored passwords.
- Forgot Password via time-limited emailed security token.

8.2 Authorization

- Strict RBAC via JWT role claims — [Authorize(Roles = "...")] on all endpoints.
- Data perimeter isolation: each user role can only access their own tenant's data.
- Prevention of privilege escalation — role claims are server-validated, not client-trusting.

8.3 Data Protection

- HTTPS encryption for all data in transit (TLS 1.2+).
- Database encryption at rest for sensitive fields.
- EF Core parameterized queries preventing SQL injection.
- Input sanitization preventing XSS attacks.
- Secure file upload with server-side validation of type, extension, and file size.

8.4 Audit & Compliance

- Comprehensive immutable AuditLog for all critical mutations.
- Audit entries capture: UserId, Timestamp, EntityName, EntityId, Action.
- Audit log retention: minimum 7 years for regulatory compliance.
- All security events (logins, privilege attempts) logged.
- Compliance targets: GDPR, PDPA, and applicable local insurance data regulations.

9. Quality Attributes

9.1 Testability

- Clean Architecture enables isolated unit testing of each layer.
- All dependencies are injected via constructor injection for easy mocking.
- xUnit test framework with Moq for mocking; EF Core In-Memory provider for integration tests.
- Minimum 80% code coverage for all critical business logic.

9.2 Maintainability

- SOLID principles applied throughout the codebase.
- Clean Architecture enforces strict layer separation and dependency direction.
- Repository + Unit of Work patterns abstract all data access.
- AutoMapper reduces DTO ↔ entity mapping boilerplate.
- Comprehensive XML documentation comments on all public interfaces.

9.3 Extensibility

- Interface-based service design allows implementation swapping without breaking callers.
- Open-Closed Principle: new coverage types, billing frequencies, or claim types can be added without modifying existing logic.
- Background worker is designed for future extension with additional daily jobs.

9.4 Performance

- Asynchronous programming (async/await) used throughout all service layers.
- Proper EF Core indexing on frequently queried foreign keys and status columns.
- Pagination enforced on all list endpoints to prevent unbounded result sets.
- Background jobs handle long-running operations asynchronously.

9.5 Availability

- Stateless API design supports horizontal scaling behind a load balancer.
- Health check endpoints for infrastructure monitoring.
- Global exception handler prevents unhandled exceptions from crashing the service.
- Database connection pooling via EF Core.

10. Appendices

10.1 API Endpoints Summary

Endpoint	Method	Role	Description
/api/auth/register	POST	Public	User registration
/api/auth/login	POST	Public	User login — returns JWT
/api/auth/admin/create-staff	POST	Admin	Create Agent or ClaimsOfficer account
/api/auth/change-password	POST	All	Change password
/api/auth/forgot-password	POST	Public	Email-based password reset
/api/admindashboard/summary	GET	Admin	System-wide metrics + analytics
/api/agentdashboard/summary	GET	Agent	Commission and customer data
/api/customerdashboard/summary	GET	Customer	Policy, member, claim overview
/api/claimsofficerdashboard/summary	GET	ClaimsOfficer	Claim queue and stats
/api/insuranceplan	POST	Admin	Create insurance plan
/api/insuranceplan	GET	All	List all plans
/api/insuranceplan/{id}	PUT	Admin	Update insurance plan
/api/corporateclient	POST	Agent	Register corporate client
/api/corporateclient/upload-document	POST	Customer	Upload KYB document
/api/corporateclient/review	POST	Admin	Approve or reject KYB
/api/policyassignment	POST	Agent	Assign plan to client
/api/member/upload	POST	Customer	Excel bulk member upload
/api/member/{policyId}	GET	All	List policy members
/api/claim/submit	POST	Customer	Submit new claim
/api/claim/take/{id}	POST	ClaimsOfficer	Lock claim for adjudication
/api/claim/release/{id}	POST	ClaimsOfficer	Release claim back to queue
/api/claim/review	POST	ClaimsOfficer	Approve or reject claim
/api/claim/pending	GET	ClaimsOfficer	List open claim queue
/api/endorsement/submit	POST	Customer	Submit endorsement request
/api/endorsement/review	POST	Agent/Admin	Approve or reject endorsement
/api/endorsement/pending	GET	Agent/Admin	List pending endorsements
/api/invoice/{policyId}	GET	All	Get policy invoices
/api/invoice/pay	POST	Customer	Submit payment for invoice
/api/notifications	GET	All	Poll notifications for current user
/api/auditlog	GET	Admin	Global audit ledger

10.2 Database Schema Summary

- Users: Id (PK), Name, Email, PasswordHash, Role, Status, RequirePasswordChange, CreatedAt
- InsurancePlans: Id (PK), PlanCode, PlanName, BasePremium, Description, IsActive
- PlanCoverages: Id (PK), InsurancePlanId (FK), CoverageType, CoverageLimit, CoveredGroup
- CorporateClients: Id (PK), CompanyName, RegistrationNumber, BusinessCategory, VerificationStatus, RejectionCount
- AgentCustomers: Id (PK), AgentUserId (FK), CorporateClientId (FK)
- PolicyAssignments: Id (PK), InsurancePlanId (FK), CorporateClientId (FK), StartDate, EndDate, BillingFrequency, Status
- Members: Id (PK), PolicyAssignmentId (FK), Name, Email, DateOfBirth, Gender, RemainingCoverage
- Dependents: Id (PK), MemberId (FK), Name, DateOfBirth, Gender, RelationshipType
- Claims: Id (PK), ClaimNumber, PolicyAssignmentId (FK), MemberId (FK), DependentId (FK), ClaimType, ClaimAmount, SanctionedAmount, Status, RejectionReason, ReviewedByUserId (FK)
- ClaimDocuments: Id (PK), ClaimId (FK), DocumentType, FilePath
- PolicyEndorsements: Id (PK), PolicyAssignmentId (FK), Type, EndorsementDataJson, Status, PremiumAdjustment, RequestedByUserId (FK), ReviewedByUserId (FK)
- Invoices: Id (PK), PolicyAssignmentId (FK), InvoiceNumber, Amount, PenaltyAmount, DueDate, Status
- Payments: Id (PK), InvoiceId (FK), Amount, PaymentDate, PaymentMethod, Status
- AuditLogs: Id (PK), UserId (FK), Action, EntityName, EntityId, Timestamp
- Notifications: Id (PK), UserId (FK), Message, IsRead, CreatedAt

10.3 Enumeration Types

Enumeration	Values
UserRole	Admin, Agent, ClaimsOfficer, Customer
UserStatus	Active, Inactive, Suspended, PermanentlyBlocked
ClaimStatus	Pending, InReview, Approved, Rejected
EndorsementStatus	Pending, Approved, Rejected
EndorsementType	AddMember, RemoveMember, AddDependent, RemoveDependent
PolicyStatus	Active, Suspended, Expired
InvoiceStatus	Pending, Paid, Overdue, Cancelled
PaymentStatus	Pending, Completed, Failed
PaymentMethod	Cash, CreditCard, DebitCard, BankTransfer, Cheque
CoverageType	Health, Life, Accident, CriticalIllness
CoveredGroup	Employee, Spouse, Children, Parents
ClaimDocumentType	MedicalBill, Prescription, LabReport, DischargeSummary, Other
DocumentType	RegistrationCertificate, TaxDocument, BusinessLicense, Other

Gender	Male, Female, Other
RelationshipType	Spouse, Child, Parent, Sibling
BillingFrequency	Monthly, Annually
BusinessCategory	IT, Manufacturing, Healthcare, Education, Finance, Other
VerificationStatus	Pending, Verified, Rejected

10.4 Design Patterns Used

- Repository Pattern: Abstracts data access; provides a collection-like interface for domain entities.
- Unit of Work Pattern: Coordinates transactional consistency across multiple repository operations.
- Dependency Injection: Constructor injection throughout, enabling loose coupling and testability.
- Strategy Pattern: Applied in premium/commission calculation — configurable rate rules without branching.
- Observer / Notification Pattern: Notification service for event-driven cross-role alerts.
- DTO Pattern: Separates internal domain models from external API contracts via AutoMapper.
- Middleware Pattern: Request/response pipeline for global exception handling and request logging.
- Concurrency Lock Pattern: Claims adjudication 'Take/Release' mechanism prevents double-processing.

10.5 Future Enhancements

- Microservices Architecture: Decompose monolith into domain-specific services (Claims, Billing, Identity).
- Cloud Storage Integration: Azure Blob Storage or AWS S3 for scalable document management.
- Real-time Notifications: Replace polling with WebSocket / ASP.NET Core SignalR.
- Payment Gateway Integration: Stripe, Razorpay, or local payment processors.
- Advanced Analytics: ML-based fraud detection and risk assessment.
- Mobile Applications: Native iOS and Android apps for agents and customers.
- Multi-tenancy: Isolated data partitioning for multiple independent insurance providers.
- Event-Driven Architecture: Message queues (RabbitMQ / Azure Service Bus) for async processing.
- Biometric Authentication: Fingerprint / face recognition for mobile apps.
- API Gateway: Centralised routing, rate limiting, and versioning.